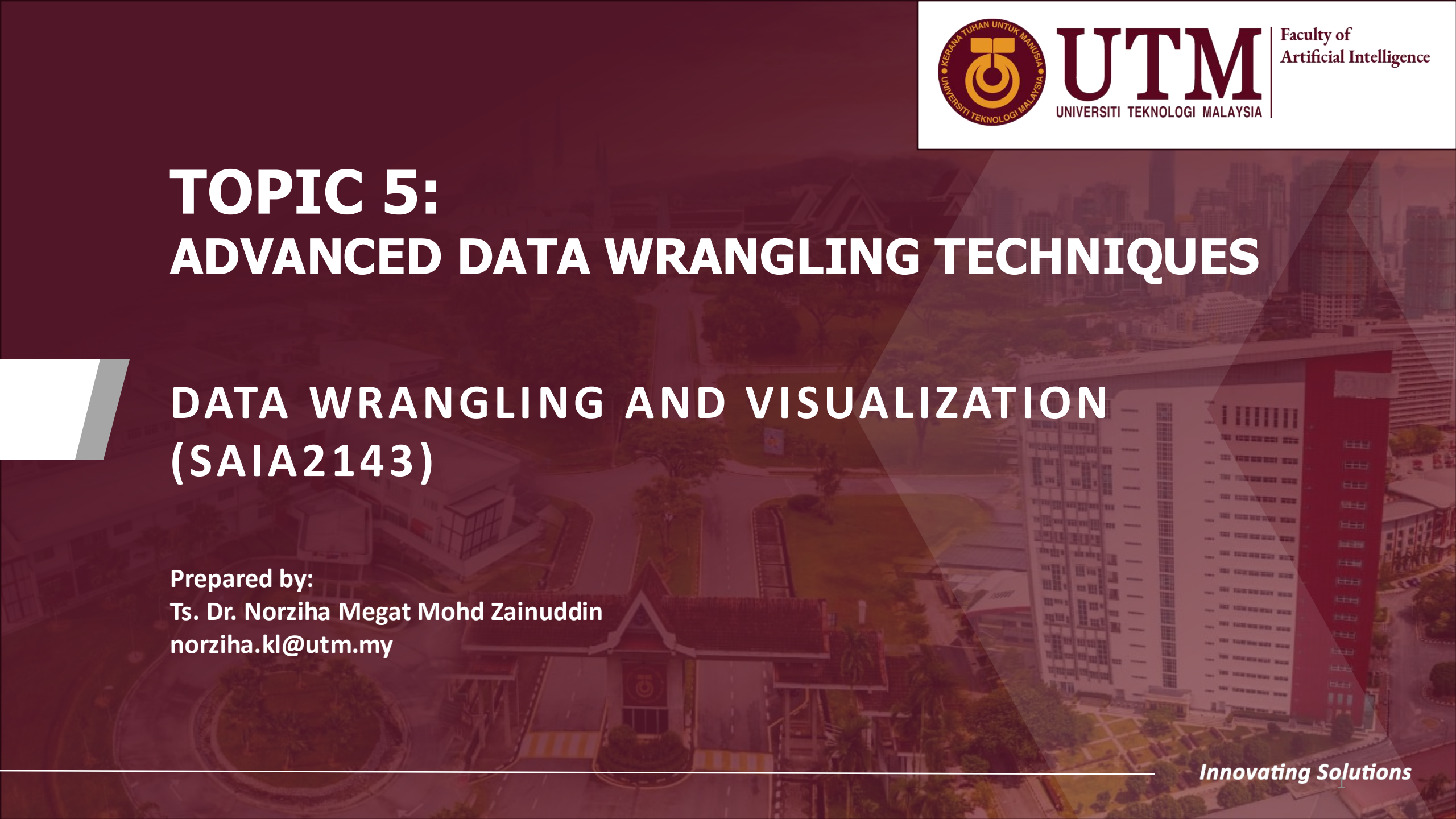


UTM
UNIVERSITI TEKNOLOGI MALAYSIA
Faculty of Artificial Intelligence

TOPIC 5:
ADVANCED DATA WRANGLING TECHNIQUES

DATA WRANGLING AND VISUALIZATION (SAIA2143)

Prepared by:
Ts. Dr. Norziha Megat Mohd Zainuddin
norziha.kl@utm.my

COURSE LEARNING OUTCOMES

At the end of this class, students should be able to:

a. Techniques for managing large datasets efficiently
b. Best practices for merging and joining multiple datasets
c. Handling time-series data and integrating external APIs

Introduction

- **Definition:** Process of cleaning, reshaping, transforming raw data into usable format.

- Data Wrangling may further include:
  a. Data Aggregation
  b. Training statical models for predictions

Topic 5: Advanced Data Wrangling Techniques

a. **Dealing with large datasets and memory management**
b. Merging and joining datasets using Pandas
c. Handling time-series data and working with external APIs

Small Data vs Large Data Challenges

- Memory limits/ memory crashes (Pandas loads entire dataset into RAM)
- Slow processing on huge files (>1GB, millions of rows)
- Complex joins from multiple sources
- Real-time or time-series data requires careful handling
- External data sources (APIs) introduce variability and scale

Strategies for large datasets

i. Use of `dtype` to control memory
ii. Chunking with `pd.read_csv(chunksize=...)`
iii. Downcasting numeric types `(astype)`
iv. Using `Dask` as a scalable alternative to Pandas

i. Use of `dtype` to control memory

- Pandas assigns default `dtypes` when loading data (e.g., int64, float64, object)
- Larger `dtypes` consume **more memory** than necessary

Data Type (`dtypes`)	Recommended dtype
Small integers	uint8, int16
Large integers	int32
Floating numbers	float32
Categorical strings	category
Dates/times	datetime64[ns]

Example

Tip: Always check `.memory_usage()` to monitor memory impact!

```python
import pandas as pd

# Default loading (no dtype control)
df = pd.read_csv('large_data.csv')

# Check memory usage
print(df.info(memory_usage='deep'))

# Manually specify smaller types
dtypes = {
    'user_id': 'int32',
    'age': 'uint8',
    'country': 'category',
    'purchase_amount': 'float32'
}

df_optimized = pd.read_csv('large_data.csv', dtype=dtypes)

# Check optimized memory usage
print(df_optimized.info(memory_usage='deep'))
```

www.fai.utm.my

Instruction:

Identify the memory used before and after using `dtype`

```python
# Google Colab Notebook: dtype Memory Optimization Demo

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time

# Step 1: Generate sample large dataset
n_rows = 1_000_000

data = {
    'user_id': np.random.randint(1, 1_000_000, n_rows),
    'age': np.random.randint(18, 90, n_rows),
    'country': np.random.choice(['US', 'CA', 'UK', 'AU'], n_rows),
    'purchase_amount': np.random.uniform(10.0, 500.0, n_rows)
}

# Create DataFrame and save to CSV
pd.DataFrame(data).to_csv('large_data.csv', index=False)
print("Sample dataset saved as 'large_data.csv'")

# Step 2: Load without dtype optimization
start_time = time.time()
df = pd.read_csv('large_data.csv')
time_no_opt = time.time() - start_time
print(f"\nWithout dtype optimization loaded in {time_no_opt:.2f} seconds")
print(df.info(memory_usage='deep'))

# Step 3: Load with dtype optimization
dtypes = {
    'user_id': 'int32',
    'age': 'uint8',
    'country': 'category',
    'purchase_amount': 'float32'
}

start_time = time.time()
df_opt = pd.read_csv('large_data.csv', dtype=dtypes)
time_opt = time.time() - start_time
print(f"\nWith dtype optimization loaded in {time_opt:.2f} seconds")
print(df_opt.info(memory_usage='deep'))

# Step 4: Compare memory usage
mem_before = df.memory_usage(deep=True).sum() / (1024 ** 2)  # MB
mem_after = df_opt.memory_usage(deep=True).sum() / (1024 ** 2)  # MB

print(f"\nMemory before optimization: {mem_before:.2f} MB")
print(f"Memory after optimization: {mem_after:.2f} MB")
print(f"Memory reduction: {100 * (1 - mem_after / mem_before):.2f}%")

# Step 5: Visualize memory usage
labels = ['Without dtype', 'With dtype']
memory = [mem_before, mem_after]

plt.bar(labels, memory, color=['red', 'green'])
plt.ylabel('Memory Usage (MB)')
plt.title('Memory Usage Before vs After dtype Optimization')
plt.show()

# Step 6: Optional: Using Dask for even larger datasets
!pip install dask[dataframe]
import dask.dataframe as dd

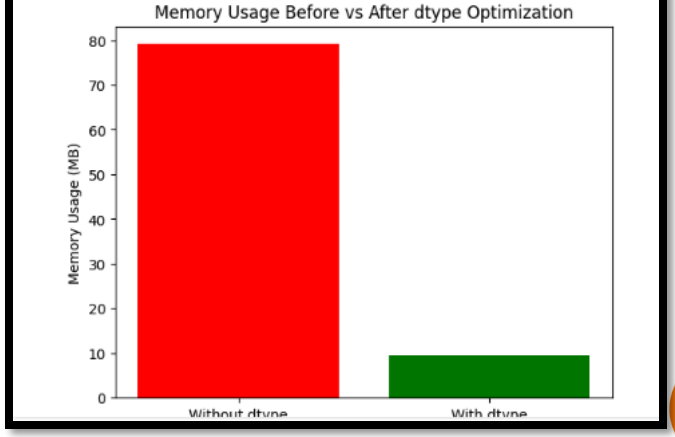
# Load using Dask (handles datasets larger than RAM)
dask_df = dd.read_csv('large_data.csv')
print("\nDask DataFrame loaded")
print(dask_df.head())
```

```
Sample dataset saved as 'large_data.csv'

Without dtype optimization loaded in 1.56 seconds
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 4 columns):
 #   Column           Non-Null Count    Dtype
---  ------           --------------    -----
 0   user_id          1000000 non-null  int64
 1   age              1000000 non-null  int64
 2   country          1000000 non-null  object
 3   purchase_amount  1000000 non-null  float64
dtypes: float64(1), int64(2), object(1)
memory usage: 79.2 MB
None

With dtype optimization loaded in 2.22 seconds
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 4 columns):
 #   Column           Non-Null Count    Dtype
---  ------           --------------    -----
 0   user_id          1000000 non-null  int32
 1   age              1000000 non-null  uint8
 2   country          1000000 non-null  category
 3   purchase_amount  1000000 non-null  float32
dtypes: category(1), float32(1), int32(1), uint8(1)
memory usage: 9.5 MB
None

Memory before optimization: 79.16 MB
Memory after optimization: 9.54 MB
Memory reduction: 87.95%
```



Google Colab file:

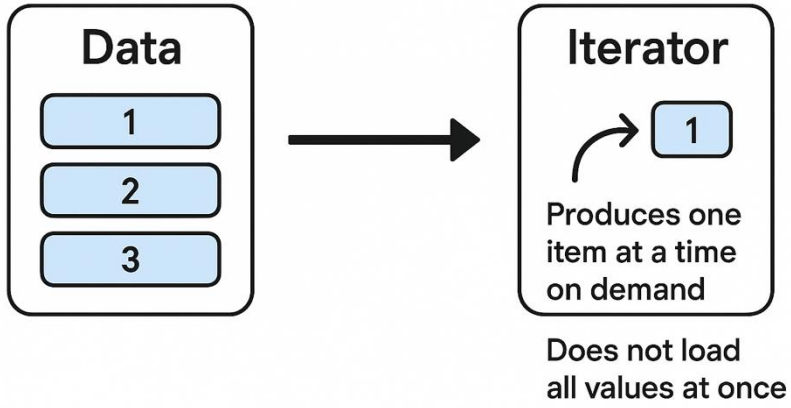
dtype Memory Optimization Demo.ipynb

ii. Chunking with `pd.read_csv(chunksize=...)`

- When you load a **huge CSV file** with Pandas, sometimes it **doesn't fit in memory (RAM)** all at once.
- To handle this, you can **read the file in chunks** — small batches of rows — using the chunksize parameter.
- This turns the read operation into an **iterator**:
  Each chunk is a smaller DataFrame you can process one at a time.

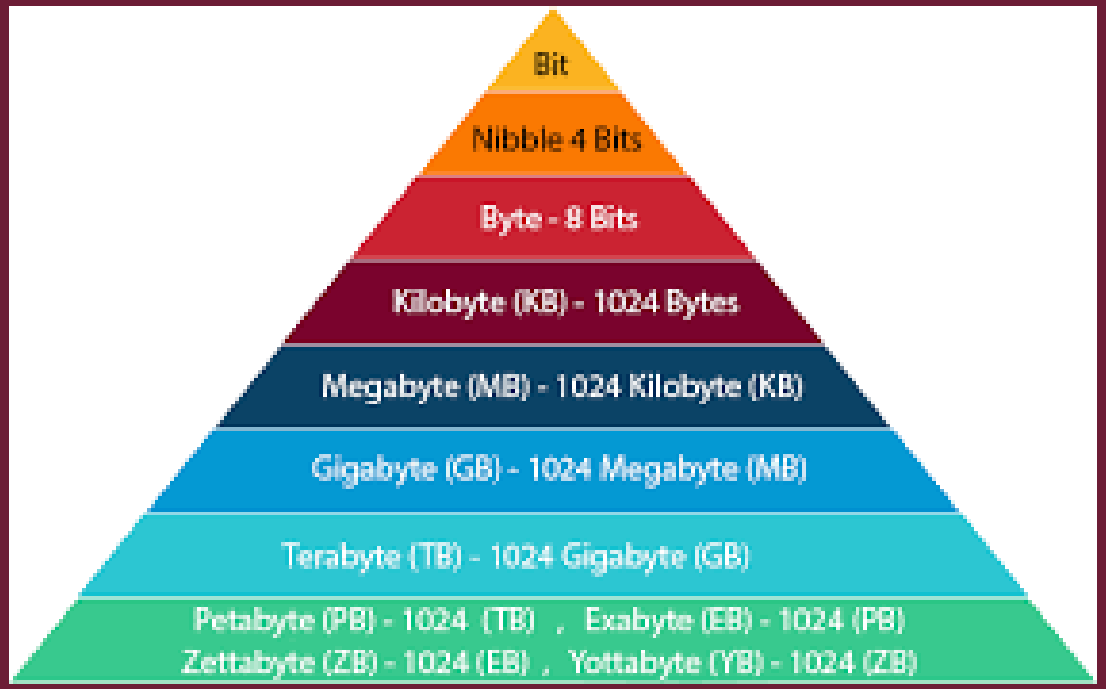
How it works?

```
for chunk in pd.read_csv('large_file.csv', chunksize=10000):
    # each 'chunk' is a DataFrame with 10,000 rows
    process(chunk)
```



When should you use chunksize?

- When files are too big for memory (≥ GB size)
- When doing aggregate operations (sum, count, mean)
- When applying transformations or filters across large data



Example 1: Sum a column across all chunks

```
import pandas as pd

total_sum = 0

for chunk in pd.read_csv('large_data.csv', chunksize=10000):
    total_sum += chunk['purchase_amount'].sum()

print(f\"Total purchase amount: {total_sum}\")
```

Accumulate the sum from each 10,000-row chunk

Example 2: Filter and save matching rows

```
chunksize = 5000
filtered_chunks = []

for chunk in pd.read_csv('large_data.csv', chunksize=chunksize):
    filtered = chunk[chunk['country'] == 'US']
    filtered_chunks.append(filtered)

# Combine all filtered chunks
final_df = pd.concat(filtered_chunks)
print(final_df.shape)
```

This filters only US rows in **batches** and combines them at the end.

Example 3: Write processed chunks to new CSV

```
for i, chunk in enumerate(pd.read_csv('large_data.csv', chunksize=10000)):
    chunk['purchase_amount'] = chunk['purchase_amount'] * 1.1  # apply 10%
    chunk.to_csv(f'processed_chunk_{i}.csv', index=False)
```

- You have 5 GB of sales data. Need to **adjust prices** or **convert currencies**.
- **Process and save** each chunk separately, avoiding large-memory operations.

  Own CSV file, like:
  - processed_chunk_0.csv
  - processed_chunk_1.csv
  - processed_chunk_2.csv
  - …

Chunked_Processing_Examples.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text

🧪 Chunked Processing Examples in Pandas

This notebook demonstrates how to use `pd.read_csv(chunksize=...)` for processing large datasets efficiently.

```
import pandas as pd
import numpy as np

# Step 1: Generate a sample large dataset
n_rows = 50000  # smaller size for demo, adjust as needed
data = {
    'user_id': np.random.randint(1, 100000, n_rows),
    'age': np.random.randint(18, 90, n_rows),
    'country': np.random.choice(['US', 'CA', 'UK', 'AU'], n_rows),
    'purchase_amount': np.random.uniform(10.0, 500.0, n_rows)
}

# Save to CSV
filename = 'large_data.csv'
pd.DataFrame(data).to_csv(filename, index=False)
print(f"✅ Sample dataset saved as '{filename}' with {n_rows} rows.")
```

```
✅ Sample dataset saved as 'large_data.csv' with 50000 rows.
```

📊 Example 1: Sum of `purchase_amount` across chunks

```
[3] total_sum = 0
    for chunk in pd.read_csv(filename, chunksize=10000):
        total_sum += chunk['purchase_amount'].sum()

    print(f"Total purchase amount: {total_sum:.2f}")
```

```
Total purchase amount: 12702572.53
```

CLASS ACTIVITY

Please run for every Examples about chunked processing.

Where can you view the output?

Google Colab file:

Chunked_Processing_Examples.ipynb

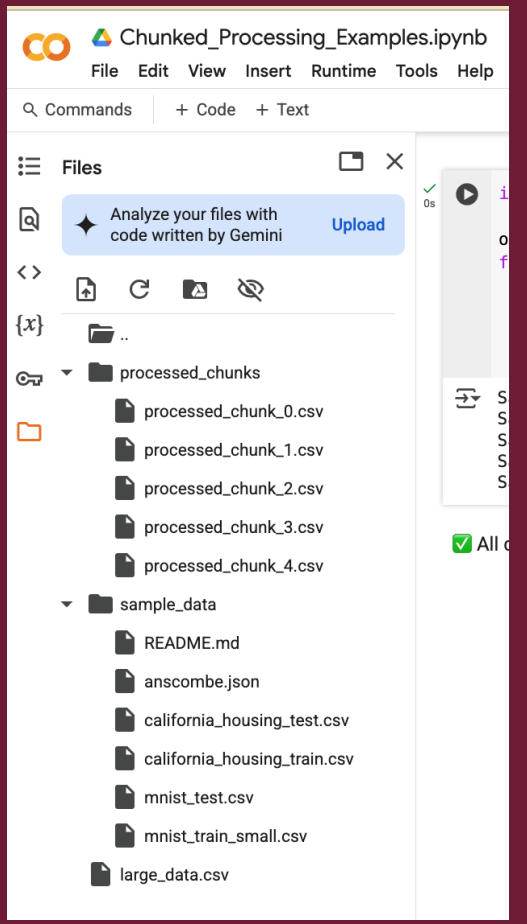
Where is the data in Google Colab?

- It's stored in the **temporary virtual environment** of Colab, specifically in the /content/ directory by default. This file is **not on your computer** unless you explicitly download it.

- **How to view or download the dataset in Colab:**
- ✅ **Option 1: Use head() to preview the data**
- df = pd.read_csv('large_data.csv') df.head()

- ✅ **Option 2: Use the Colab file browser**

1. On the left sidebar in Colab, click the **folder icon** 📂 (File Browser).
2. You'll see large_data.csv listed there.
3. Right-click and select **"Download"** to save it to your computer.



More Tips

- You can **combine all chunks** at the end using **pd.concat()**.
- You can **parallelize** chunk processing using libraries like **joblib** or **concurrent.futures** for faster performance.
- Always test with small chunks before scaling up.

iii. Downcasting numeric types (`astype`)

- Large datasets often use default data types (int64, float64).
- These types consume more memory than necessary.
- Downcasting can significantly reduce memory usage with no data loss (if used properly).

Type	Memory (Bytes)
int64	8
int32	4
int16	2
int8	1
float64	8
float32	4

Example

Expect memory usage to drop by ~50% or more.

```
import pandas as pd
import numpy as np

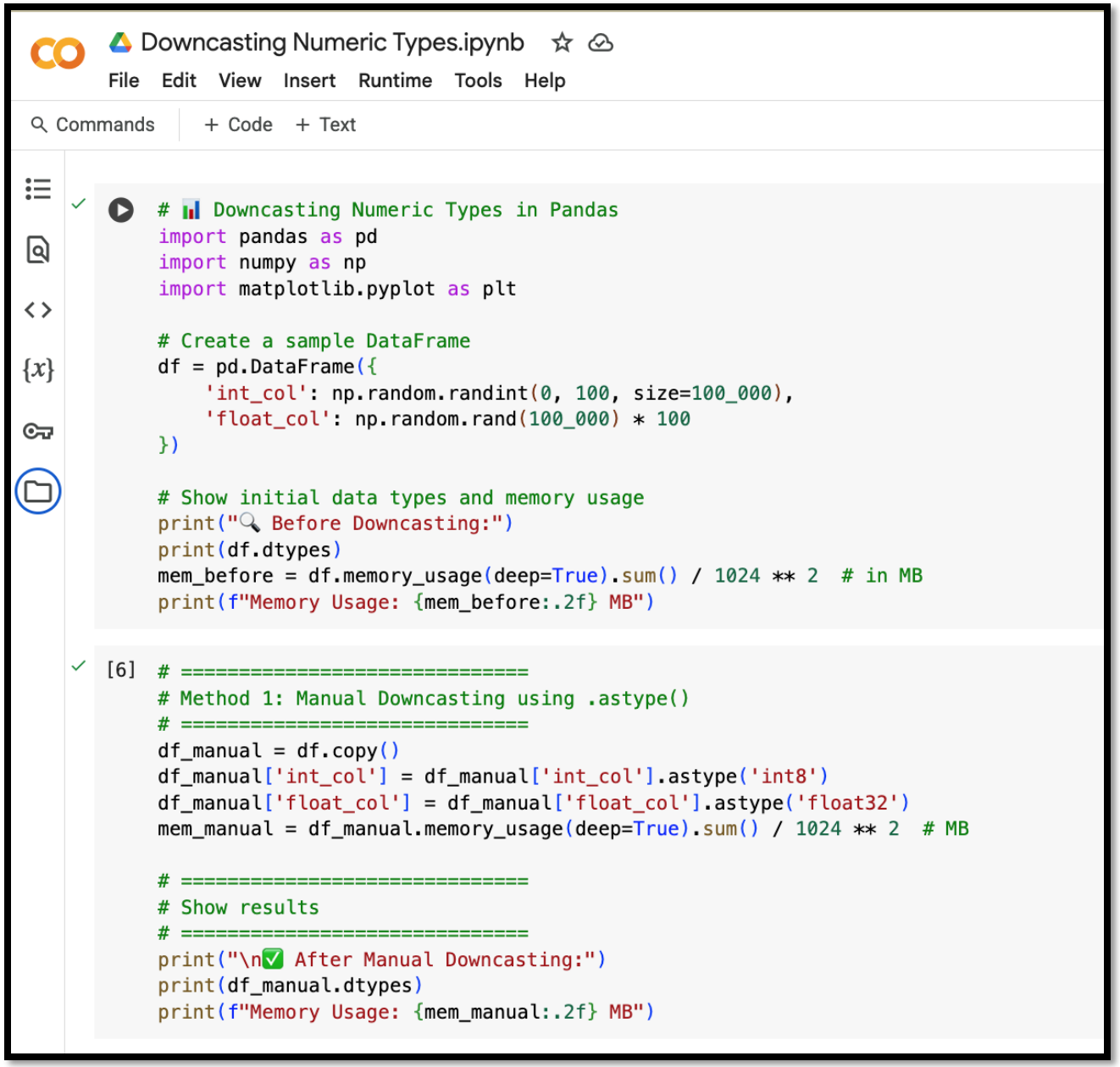
# Create a DataFrame with large numeric types
df = pd.DataFrame({
    'int_col': np.random.randint(0, 100, size=100000),
    'float_col': np.random.rand(100000) * 100
})

print("Before downcasting:")
print(df.dtypes)
print(df.memory_usage(deep=True))

# Downcast numeric columns
df['int_col'] = df['int_col'].astype('int8')
df['float_col'] = df['float_col'].astype('float32')

print("\nAfter downcasting:")
print(df.dtypes)
print(df.memory_usage(deep=True))
```

www.fai.utm.my



CLASS ACTIVITY

What is the value before and after the Downcasting?

Google Colab file:

Downcasting_Numeric_Types.ipynb.

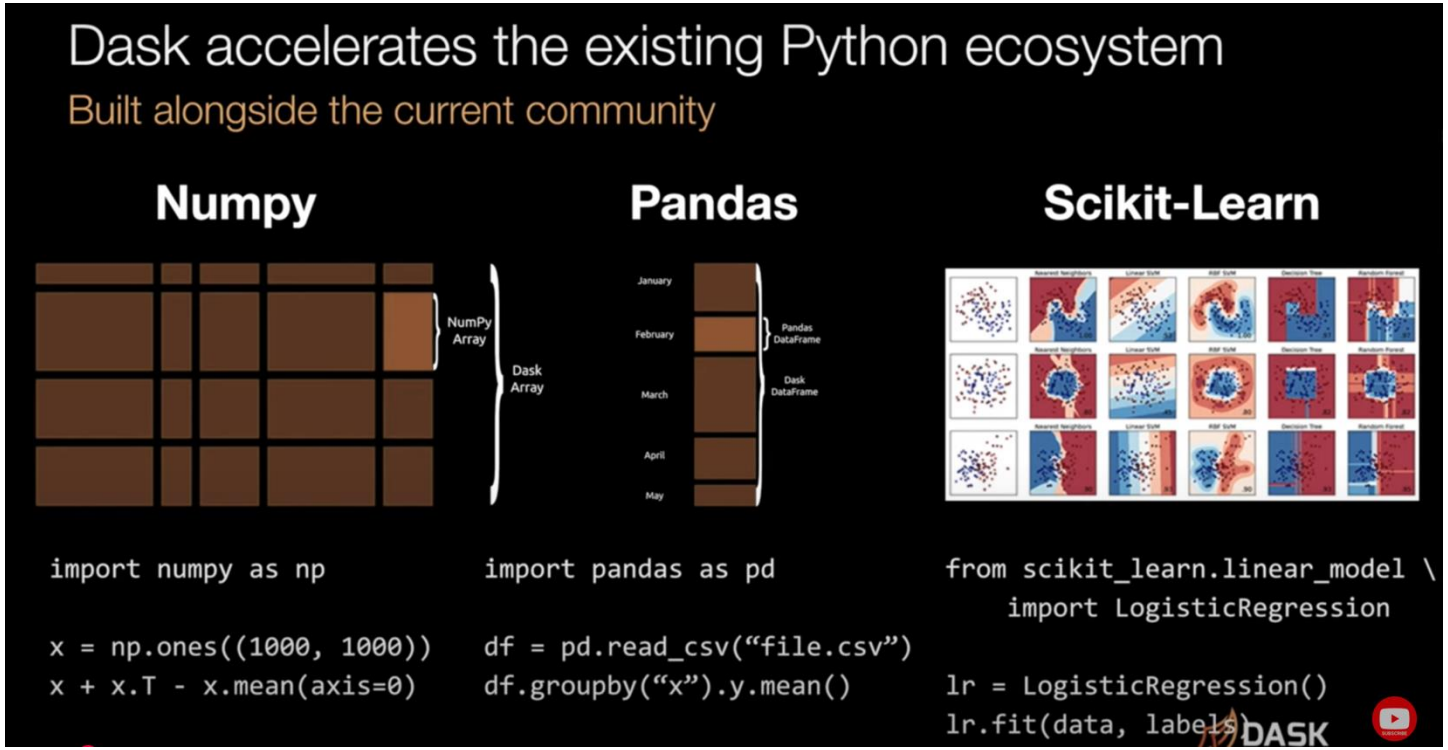
iv. Using `Dask` as a scalable alternative to Pandas

- Pandas is an excellent tool for data manipulation and analysis; however, it operates entirely in-memory and on a single CPU core.
- When working with large datasets that exceed available RAM or require faster computations across multiple cores or nodes, Pandas becomes a bottleneck.

Where usually we use Dask?

- Large Datasets That Don't Fit in RAM
- Parallel Processing on Multi-Core Machines
- Distributed Computing on Clusters
- Interactive Analysis with Scalable Dashboards
- Incremental or Streaming Data Workflows

iv. Using Dask as a scalable alternative to Pandas



Instruction:

Please understand the 8 minutes video and answer the questions in Class Activity in E-learning

YouTube link (7:19 min):

https://www.youtube.com/watch?v=nnndxbr_Xq4&t=8s

Topic 5: Advanced Data Wrangling Techniques

a. Dealing with large datasets and memory management

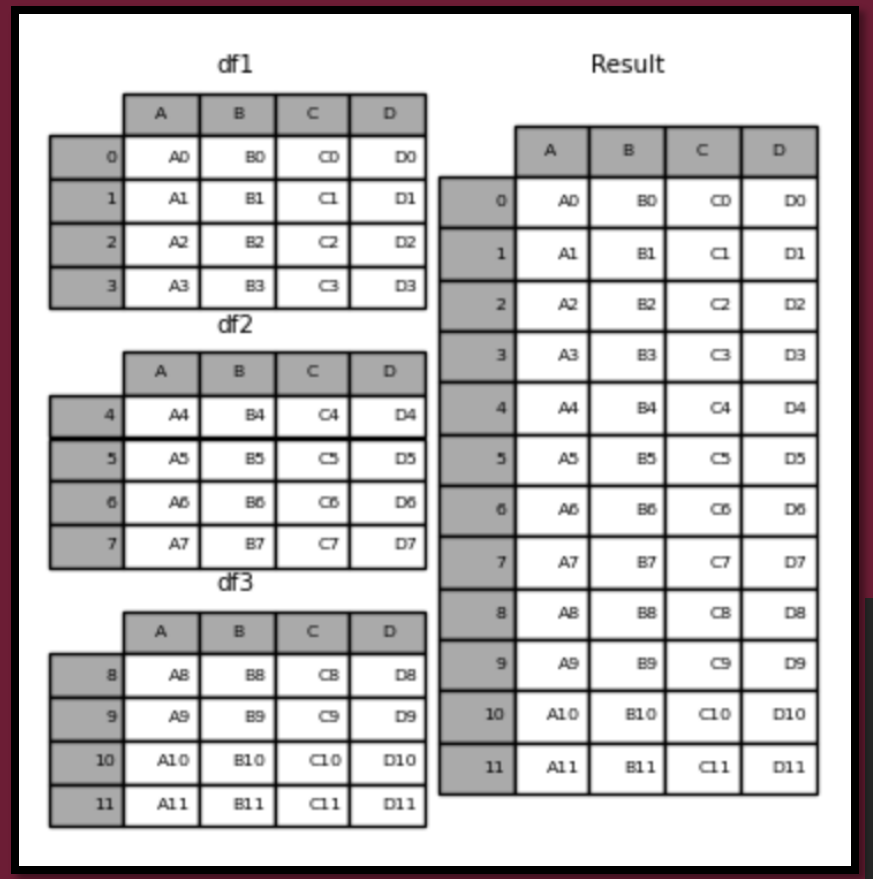
b. Merging and joining datasets using Pandas

c. Handling time-series data and working with external APIs

Data Merging and Joining

- In data analysis, it is common to combine data from multiple sources.
- Pandas provides powerful tools to perform database-style operations such as merging, joining, and concatenating DataFrames.





Key Functions Data Merging and Joining

i. **pd.merge():** Combines DataFrames based on common columns or indices.

ii. **pd.DataFrame.join():** Combines columns of another DataFrame, generally using the index.

iii. **pd.concat():** Stacks DataFrames vertically or horizontally.

i. Merging DataFrames with pd.merge()

Syntax:

```
pd.merge(left, right, how='inner', on=None, left_on=None,
```

Parameters:

- how: Type of merge – 'inner', 'outer', 'left', 'right'.
- on: Column(s) to join on (must be present in both DataFrames).

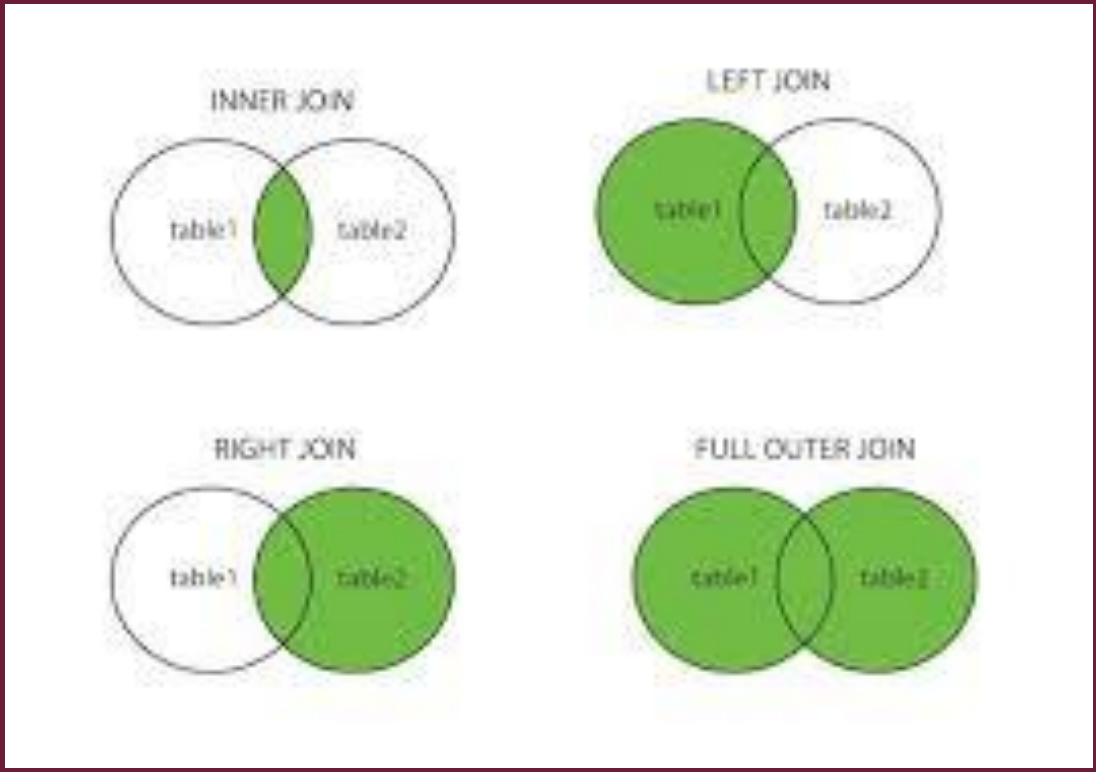
Example:

```
import pandas as pd

df1 = pd.DataFrame({'id': [1, 2, 3], 'name': ['Alice', 'Bo
df2 = pd.DataFrame({'id': [2, 3, 4], 'score': [85, 90, 88]

result = pd.merge(df1, df2, on='id', how='inner')
```

ii. Join Types

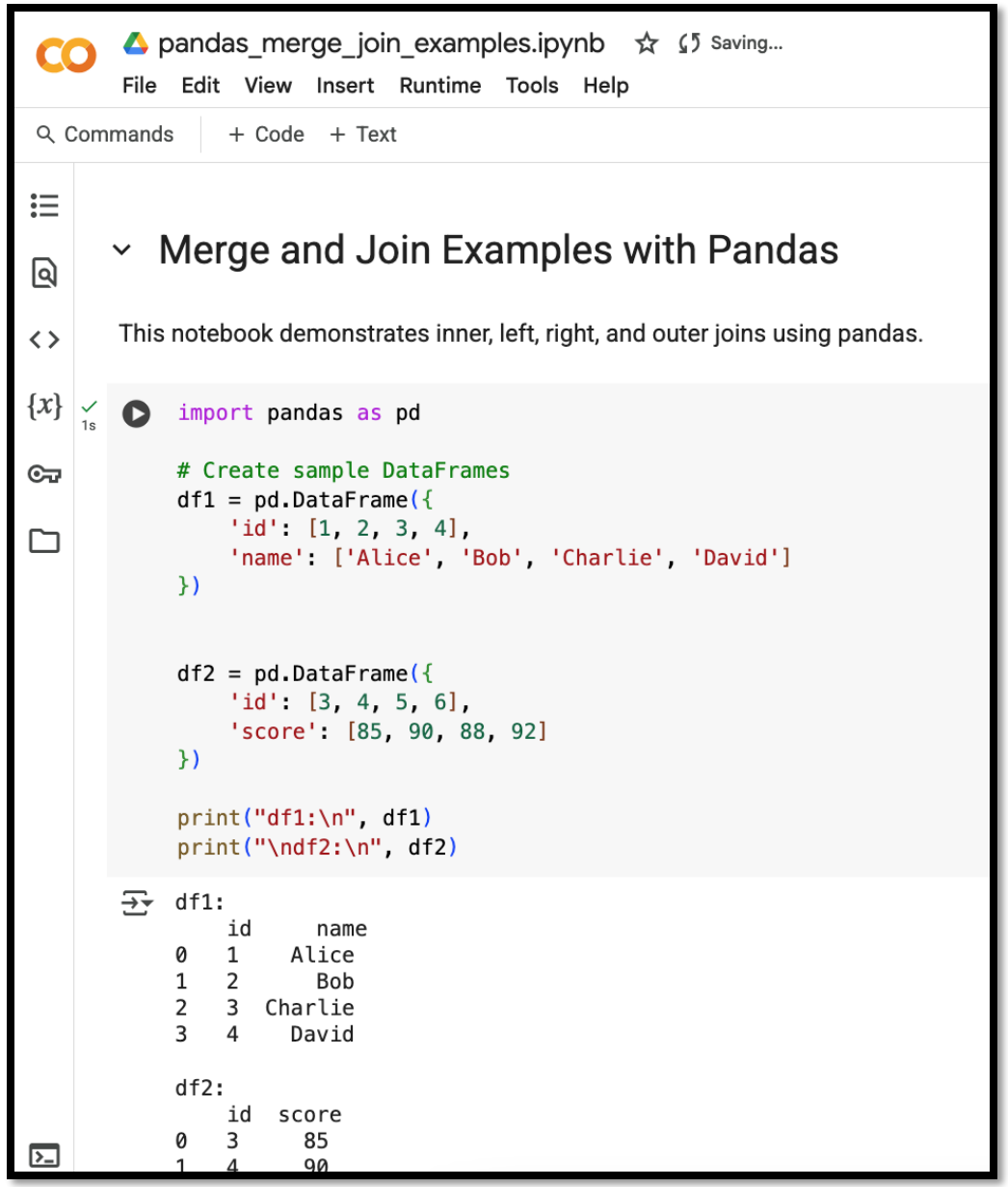


- **Inner Join**: Returns only matching rows.
- **Left Join**: Returns all rows from the left DataFrame and matched rows from the right.
- **Right Join**: Returns all rows from the right and matched rows from the left.
- **Outer Join**: Returns all rows from both DataFrames with NaNs where no match exists.

iii. Concatenating DataFrames with pd.concat()

```
frames = [df1, df2]
result = pd.concat(frames, axis=0)  # vertical
result = pd.concat(frames, axis=1)  # horizontal
```

Use ignore_index=True for reindexing when concatenating vertically.



CLASS ACTIVITY

Work with your friends. Please run the provided files, observe the outputs, and draw conclusions. Then, explain to the class the differences among the four types of **DataFrame.join()** and the use cases of **pd.concat()**.

Google Colab file:

pandas_merge_join_examples.ipynb.

Handling Key Mismatches and Suffixes

When merging, it's important to ensure the **join keys match** in:

- **Data type** (e.g., both are int64 or both are object)
- **Value** (no typos or unexpected nulls)

Handling Key Mismatches and Suffixes

Detect key mismatches:

```
print(df1.dtypes)
print(df2.dtypes)
print(df1.isnull().sum())
print(df2.isnull().sum())
```

Fix key mismatches:

```
df1['id'] = df1['id'].astype(int)
df2['id'] = df2['id'].astype(int)
df1 = df1.dropna(subset=['id'])
```

Handling Overlapping Column Names with suffixes

- If both DataFrames have columns with the same name (besides the key), Pandas will raise an error or overwrite them.
- This is essential when merging datasets with similarly named fields like "score", "status", etc.
- Use the suffixes parameter to differentiate:

```
merged = pd.merge(df1, df2, on='id', suffixes=('_df1', '_df2'))
```

Multi-Key Joins

- You can merge on **multiple keys** (composite keys) by passing a list of column names to **on**, **left_on**, and **right_on**.
- Use multi-key joins when:
  a. There's **no unique single column** identifier.
  b. You want **greater precision** in matching rows.
- Only rows where **both** **id** and **dept** match will be included.

```
df1 = pd.DataFrame({
    'id': [1, 2, 3],
    'dept': ['HR', 'IT', 'Finance'],
    'name': ['Alice', 'Bob', 'Charlie']
})

df2 = pd.DataFrame({
    'id': [1, 2, 3],
    'dept': ['HR', 'IT', 'HR'],
    'score': [90, 85, 88]
})

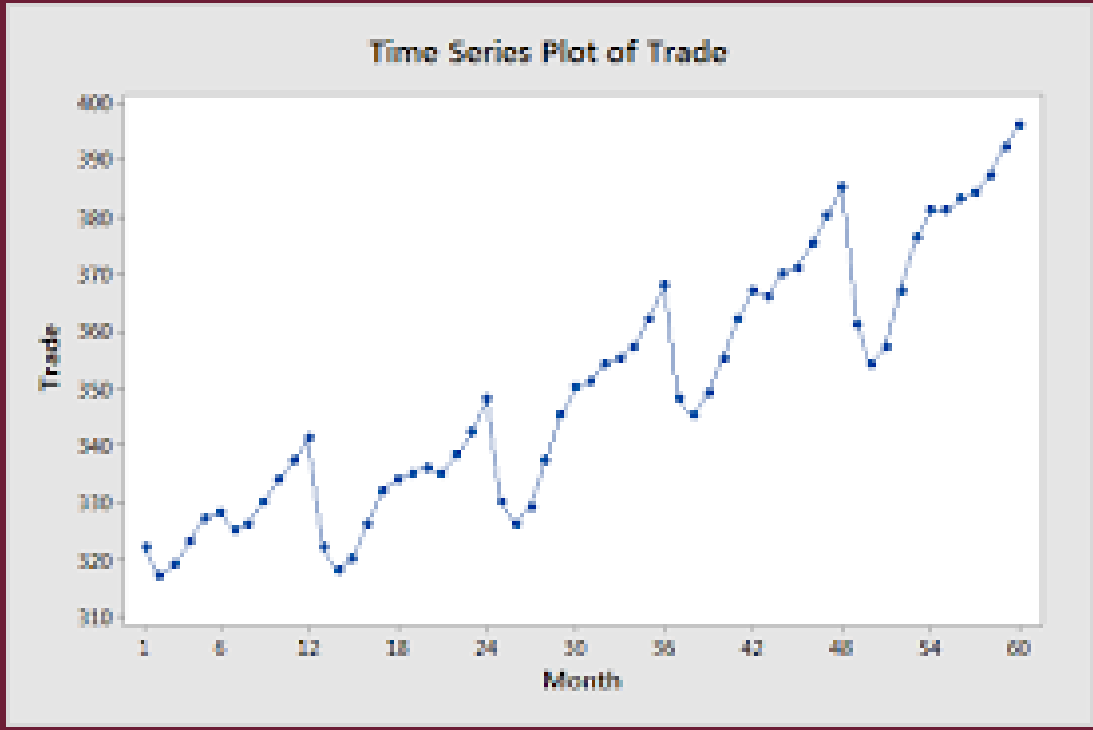
result = pd.merge(df1, df2, on=['id', 'dept'], how='inner')
```

Topic 5: Advanced Data Wrangling Techniques

a. Dealing with large datasets and memory management
b. Merging and joining datasets using Pandas
c. **Handling time-series data and working with external APIs**

What is Time-Series Data?

- Time series data is a sequence of data points collected or recorded at regular intervals over a period of time.
- It is vital for trend detection and forecasting.
- Examples include stock prices, temperature logs, sales data, etc.



Time-series Handling

i. Converting to DatetimeIndex

ii. Resampling (daily → monthly)

iii. Rolling statistics (moving averages)

i. Converting to_datetime Index

- Many datasets include a column with date/time information.
- Use **pd.to_datetime** to convert string to datetime.
- Enables time-based slicing and resampling
- Must-have for time-series functions in Pandas

```
import pandas as pd

# Sample data
data = {'date': ['2022-01-01', '2022-01-02', '2022-01-03']
        'value': [10, 15, 20]}
df = pd.DataFrame(data)

# Convert to datetime
df['date'] = pd.to_datetime(df['date'])

# Set as index
df.set_index('date', inplace=True)
print(df)
```

ii. Resampling Time-Series Data

- Resampling aggregates data to a new frequency (e.g., daily to monthly).
- `'M' = Monthly, 'D' = Daily, 'W' = Weekly`, etc.
- Use **.resample()** followed by an aggregation function (mean, sum, etc).

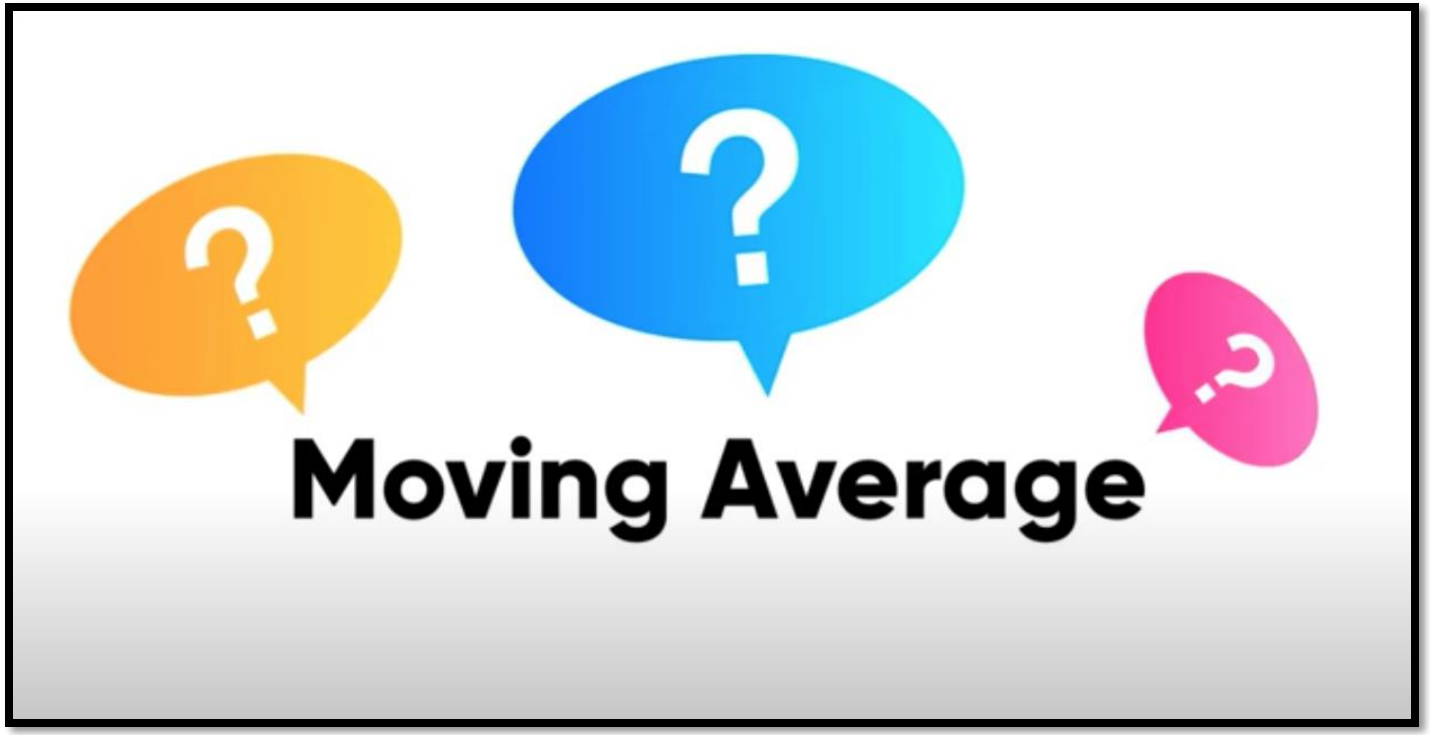
```
# Monthly average
monthly = df.resample('M').mean()
print(monthly)
```

iii. Rolling Statistics (Moving Averages)

- Useful for smoothing data and analyzing trends.
- Computes moving average over a defined window.
- Helps in smoothing fluctuations.
- Use **.rolling(window_size).mean()**

```
# 2-day moving average
rolling_avg = df.rolling(2).mean()
print(rolling_avg)
```

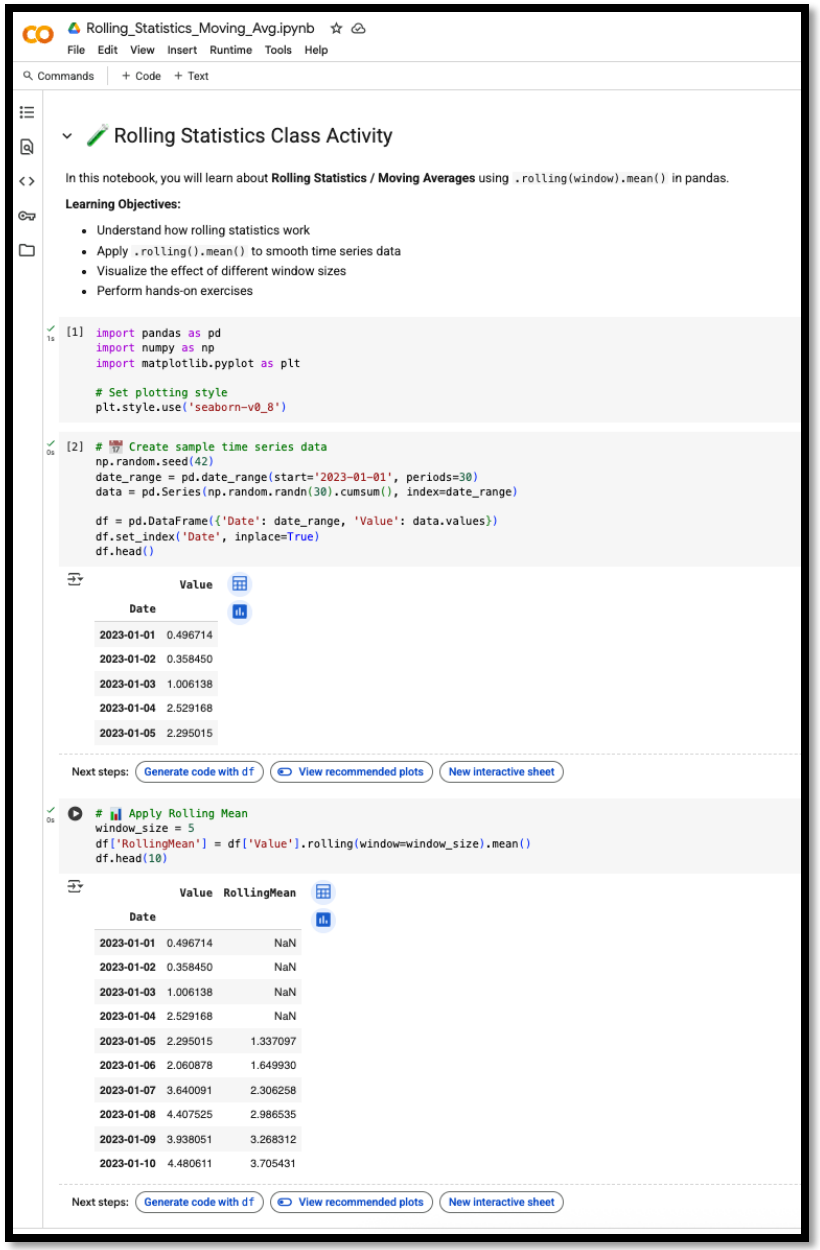
CLASS ACTIVITY



YouTube link (6:37):
https://www.youtube.com/watch?v=PuXLgZCTh5c

Instruction:

- Briefly review the concept of moving averages and their purpose in smoothing data to identify trends.
- Discuss real-life applications, such as in weather forecasting.



Rolling_Statistics_Moving_Avg.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text

Rolling Statistics Class Activity

In this notebook, you will learn about **Rolling Statistics / Moving Averages** using `.rolling(window).mean()` in pandas.

Learning Objectives:

- Understand how rolling statistics work
- Apply `.rolling().mean()` to smooth time series data
- Visualize the effect of different window sizes
- Perform hands-on exercises

```
[1] import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt

    # Set plotting style
    plt.style.use('seaborn-v0_8')
```

```
[2] # 📅 Create sample time series data
    np.random.seed(42)
    date_range = pd.date_range(start='2023-01-01', periods=30)
    data = pd.Series(np.random.randn(30).cumsum(), index=date_range)

    df = pd.DataFrame({'Date': date_range, 'Value': data.values})
    df.set_index('Date', inplace=True)
    df.head()
```

Date	Value
2023-01-01	0.496714
2023-01-02	0.358450
2023-01-03	1.006138
2023-01-04	2.529168
2023-01-05	2.295015

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
# 📊 Apply Rolling Mean
window_size = 5
df['RollingMean'] = df['Value'].rolling(window=window_size).mean()
df.head(10)
```

Date	Value	RollingMean
2023-01-01	0.496714	NaN
2023-01-02	0.358450	NaN
2023-01-03	1.006138	NaN
2023-01-04	2.529168	NaN
2023-01-05	2.295015	1.337097
2023-01-06	2.060878	1.649930
2023-01-07	3.640091	2.306258
2023-01-08	4.407525	2.986535
2023-01-09	3.938051	3.268312
2023-01-10	4.480611	3.705431

Next steps: Generate code with df | View recommended plots | New interactive sheet

CLASS ACTIVITY

Instruction:

Let's run this code and visualise the output for the moving average.

Google Colab file:

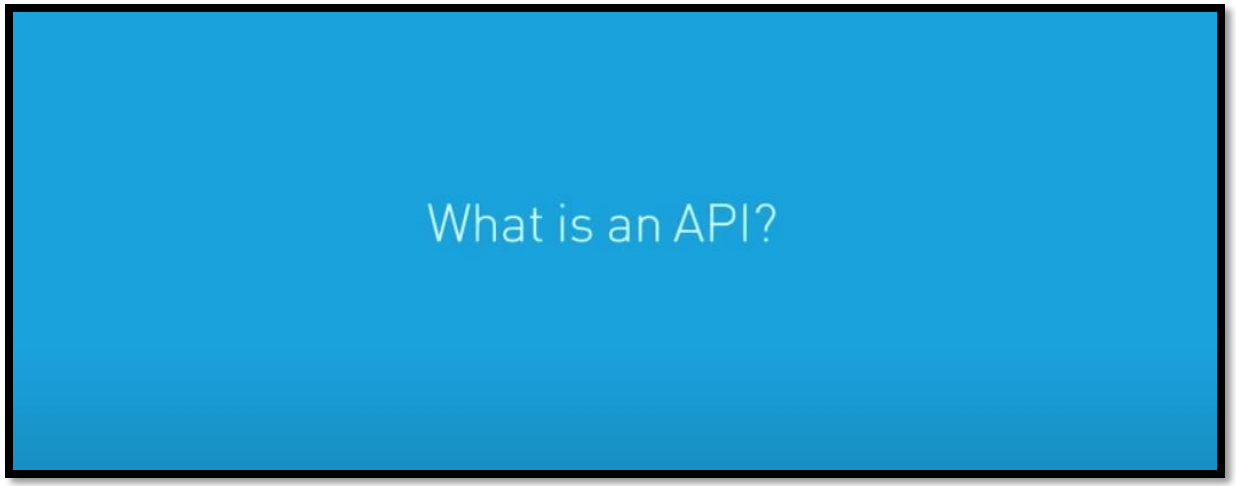
Rolling_Statistics_Moving_Avg.ipynb.

What is an External APIs?

- APIs (Application Programming Interfaces) allow data interact with external services.

- Examples: OpenWeatherMap, CoinGecko, Alpha Vantage

CLASS ACTIVITY



Instruction:

After view the YouTube video. Give one example where you can find API in your daily life.

YouTube link (3:24min):

https://www.youtube.com/watch?v=s7wmiS2mSXY

Why Use APIs in Data Analysis?

- Access **real-time**, **dynamic** data
- Automate data collection from multiple sources
- **Common Tools in Python:**
  a. requests – General-purpose HTTP library
  b. pandas.read_json() – Quick way to read JSON data directly into DataFrames

Pulling API Data with `requests`

Example: Get Cryptocurrency Prices

```
import requests

url = "https://api.coindesk.com/v1/bpi/currentprice.json"
response = requests.get(url)
data = response.json()

print(data['bpi']['USD']['rate'])
```

🔍 **Key Steps:**

1.Make a GET request

2.Convert response to JSON

3.Extract the needed info

💡 **Tip:** Use `response.status_code` to check for errors

Quick Import with `pandas.read_json()`

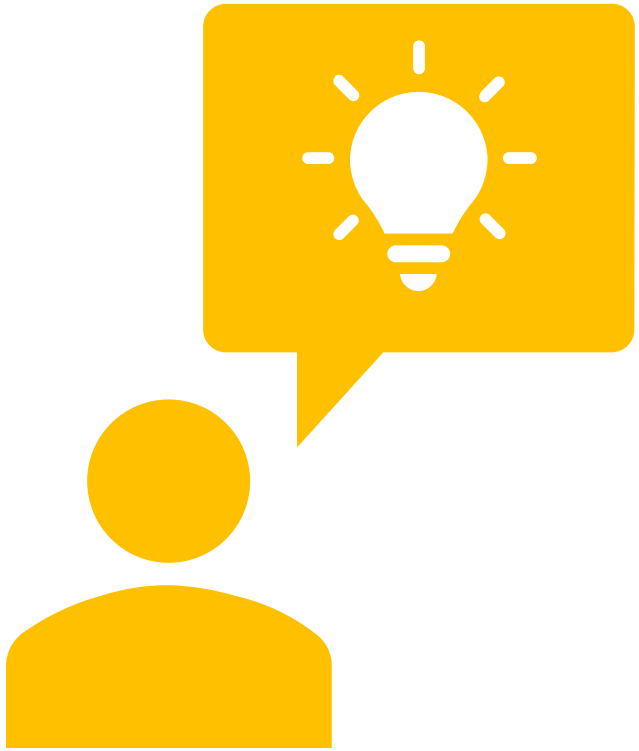
Example: Pull JSON Data into DataFrame

When to Use:

- API directly returns a structured JSON list/table
- You want to quickly explore or analyze data
- For nested JSONs, you may still need to normalize it using:

```
pd.json_normalize(data)
```

KNOWLEDGE CHECK



THANK YOU



Faculty of Artificial Intelligence (FAI)



http://fai.utm.my



Faculty of Artificial Intelligence, UTM



utmfai